

Tópicos Especiais em Linguagem de Programação

Phd Márcio Teixeira Oliveira

Ementa:

Ementa: Desenvolvimento de aplicações utilizando Arquitetura Orientada a Serviço (SOA – Service-oriented Architecture). Integração de Sistemas utilizando Web Services. Desenvolvimento de Aplicações Web com REST. Coordenação de serviços com BPEL



Bibliografia

BIBLIOGRAFIA BÁSICA

CARLSON, David. Modelagem de aplicações XML com UML. São Paulo: Makron Books, 2002.

GOMES, Daniel Adorno. Web Services Soap em Java: guia prático para o desenvolvimento de Web Services em Java. São Paulo: Novatec, 2010.

KALIN, Martin. Java Web Services: implementando. Rio de Janeiro: Alta Books, 2010.

BIBLIOGRAFIA COMPLEMENTAR

DEITEL, Paul; DEITEL, Harvey. Java: como programar. 8. ed. São Paulo: Pearson, 2010.

LUCKOW, Décio Heinzelmann; MELO, Alexandre Altair de. Programação Java para a Web: aprenda a desenvolver uma aplicação financeira pessoal com as ferramentas mais modernas da plataforma Java. São Paulo: Novatec, 2012.

MENDES, Douglas Rocha. Programação Java com ênfase em orientação a objetos. São Paulo:

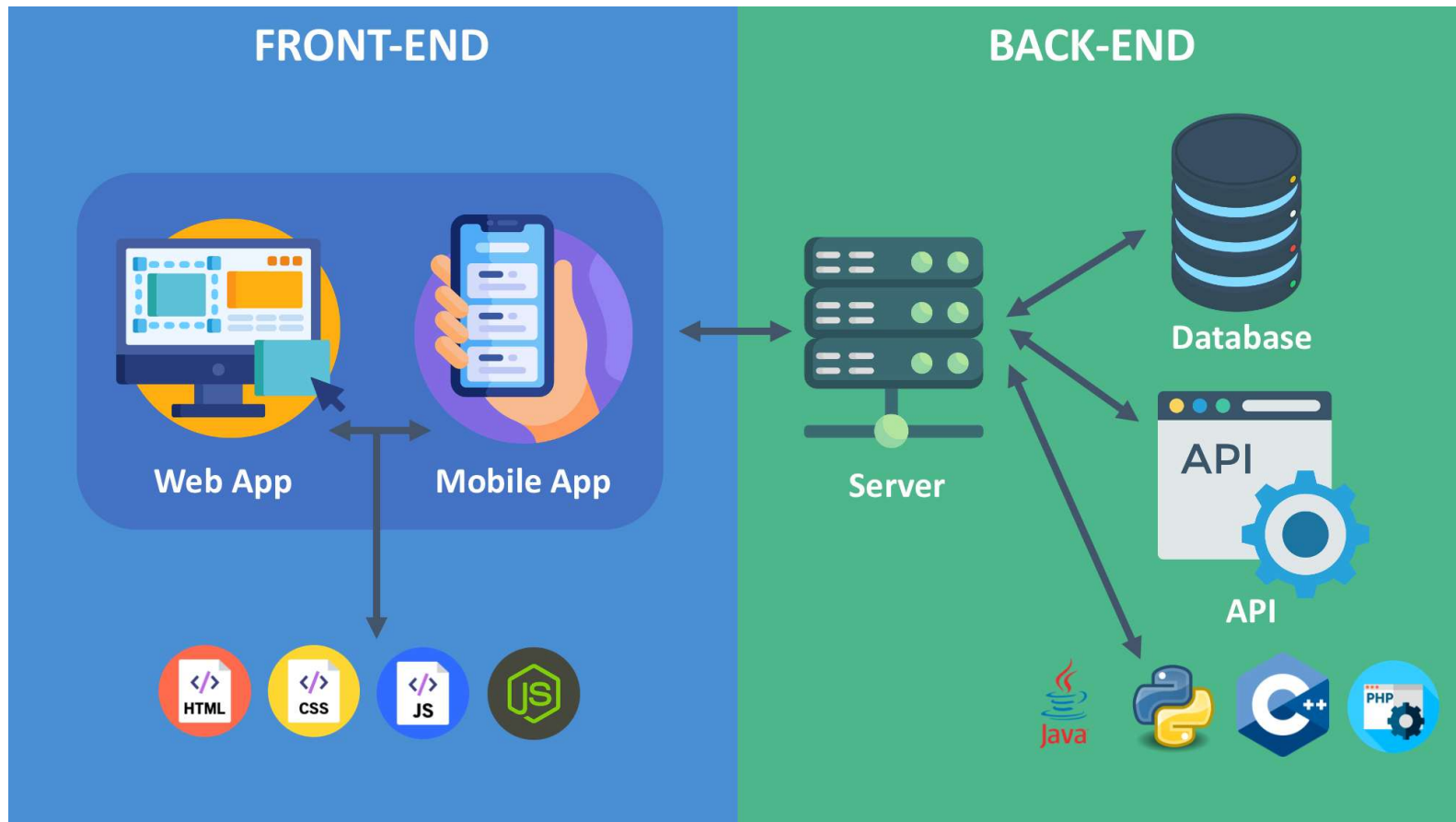


Framework

O que é?

“Framework é um conjunto de classes , métodos , funções e documentação , agrupa- dos logicamente para facilitar o desenvolvimento de programas” – Stephen G. Kochan

Arquitetura de serviço

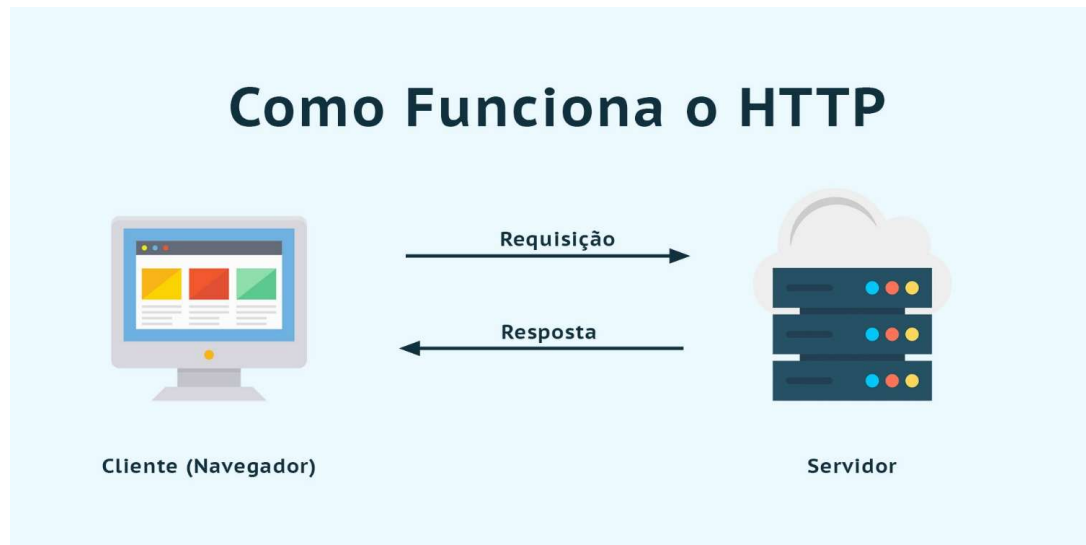


Framework – Spring

Spring é um framework web minimalista, flexível e de código aberto para Java, projetado para facilitar a criação de APIs, aplicações web e serviços backend de alto desempenho.



Protocolo HTTP



Cliente (Navegador)

É o computador do usuário — por exemplo:

- Browser
- Aplicativo mobile
- Sistema que consome API

O cliente **inicia a comunicação**.

Exemplo: você digita `https://google.com`

Servidor

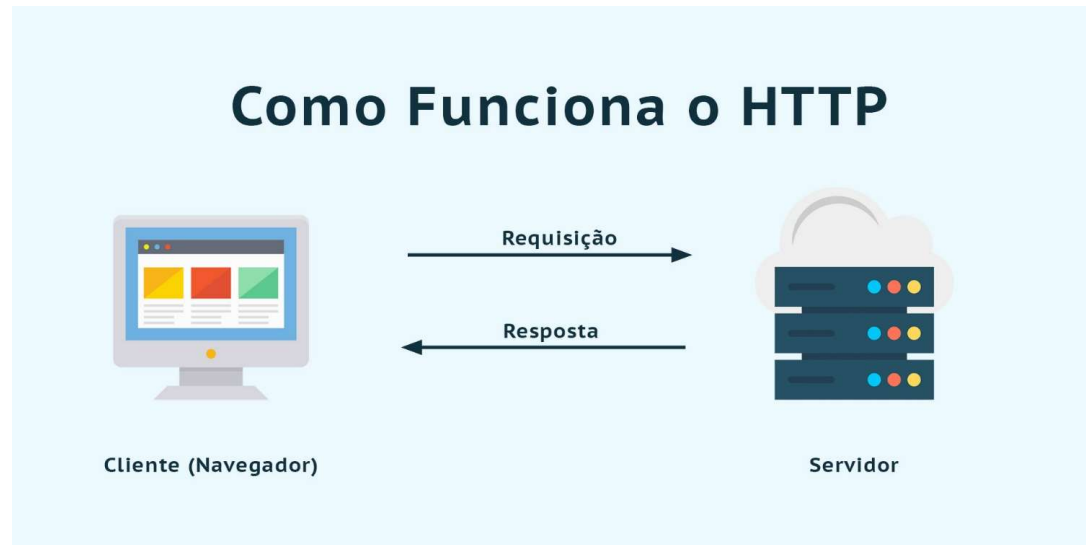
É o computador remoto que hospeda o site ou sistema.

Pode ser:

- Servidor web (Apache, Nginx)
- Backend (Spring Boot, Node.js, Django etc.)
- API REST
- Serviços em nuvem (AWS, Azure, etc.)

O servidor **processa a solicitação e responde**.

Protocolo HTTP



Fluxo de comunicação HTTP

Requisição (Request)

Seta da esquerda para a direita.

O cliente envia uma solicitação ao servidor contendo:

- URL solicitada
- Método HTTP (**GET, POST, PUT, DELETE...**)
- Cabeçalhos (headers)
- Dados (body), se houver

Resposta (Response)

Seta da direita para a esquerda.

O servidor devolve ao cliente:

- Código de status (200, 404, 500...)
- Cabeçalhos
- Conteúdo solicitado (HTML, JSON, imagem etc.)

📌 Exemplo:

HTTP/1.1 200 OK

Content-Type: application/json

Criar um novo projeto utilizando Spring Boot

Entrar pelo navegador em <https://start.spring.io/>



Criar um novo projeto utilizando Spring Boot

The screenshot displays the Spring Initializr web interface. On the left, a sidebar contains a hamburger menu icon and a refresh icon. The main content area is divided into several sections:

- Project:** Includes radio buttons for **Gradle - Groovy**, **Gradle - Kotlin**, **Java** (selected and circled in red), **Kotlin**, and **Groovy**. Below this, the **Maven** option is also circled in red.
- Spring Boot:** Includes radio buttons for versions **4.1.0 (SNAPSHOT)**, **4.1.0 (M2)**, **4.0.4 (SNAPSHOT)**, **4.0.3** (selected), and **3.5.12 (SNAPSHOT)**, **3.5.11**.
- Project Metadata:** A red box highlights the **Group** field (br.com.ifms), **Artifact** field (demo), **Name** field (Projeto), **Description** field (Projeto), and **Package name** field (br.com.ifms.demo).
- Packaging:** Includes radio buttons for **Jar** (selected and circled in red) and **War**.
- Configuration:** Includes radio buttons for **Properties** (selected) and **YAML**.
- Java:** Includes radio buttons for versions **25**, **21**, and **17** (selected).

On the right side, the **Dependencies** section features a button **ADD DEPENDENCIES... CTRL + B**. Below this, several dependencies are listed, each with a category tag and a description:

- Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
- Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Spring Boot DevTools** (DEVELOPER TOOLS): Provides fast application restarts, LiveReload, and configurations for enhanced development experience.
- MySQL Driver** (SQL): MySQL JDBC driver.
- Lombok** (DEVELOPER TOOLS): Java annotation library which helps to reduce boilerplate code.

At the bottom, there are three buttons: **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and an ellipsis button **...**.

Arquitetura Mínima Spring - Controller

A **classe Controller** no Spring (especialmente no **Spring Boot / Spring MVC**) é responsável por **receber requisições HTTP do cliente (navegador, app, API)**, processá-las e devolver uma resposta.

Spring – Controller Exemplo

A anotação **@RestController** no Spring Boot indica que a classe é um **Controller REST**, ou seja, uma classe que expõe endpoints de API e retorna dados diretamente (geralmente em **JSON**).

@RestController . . .

```
public class MeuPrimeiroController {
```

```
}
```

Spring – @GetMapping

```
@RestController
public class MeuPrimeiroController {

    @GetMapping("/Mensagem")
    public String home() {
        return "Olá ";
    }
}
```

Indica que um método do Controller será executado quando a aplicação receber uma requisição HTTP do tipo **GET** para o caminho **/Mensagem**. – <http://localhost:8080/Mensagem>

Spring – @GetMapping com parâmetros - @RequestParam

```
@RestController  
public class MeuPrimeiroController {
```

```
    @GetMapping("/Mensagem1")  
    public String mensagem1() {  
        return "Olá ";  
    }
```

```
    @GetMapping("/Mensagem2")  
    public String mensagem2(@RequestParam String nome) {  
        return "Olá, " + nome;  
    }  
}
```

Um método **GET com parâmetro** permite enviar dados pela URL para o servidor. Para acessar basta usar a URL:

<http://localhost:8080/mensagem?nome=Marcio>

Spring – @GetMapping com parâmetros - @PathVariable

```
@RestController  
public class MeuPrimeiroController {
```

```
    @GetMapping("/Mensagem1")  
    public String mensagem1() {  
        return "Olá ";    }
```

```
    @GetMapping("/Mensagem2")  
    public String mensagem2(@RequestParam String nome) {  
        return "Olá, " + nome;}
```

```
    @GetMapping("/Mensagem3/{nome}")  
    public String mensagem3(@PathVariable String nome) {  
        return "Olá, " + nome;}}
```

A anotação @PathVariable no Spring é usada para capturar valores que vêm diretamente no caminho da URL (path).:
<http://localhost:8080/mensagem3/márcio>



Classe beans

JavaBeans são classes Java padrão, serializáveis e reutilizáveis, utilizadas para encapsular dados em um único objeto

Exemplo classe

```
public class Usuario{  
  
    long idUsuario;  
    String nome;  
    String senha;  
  
}
```



Exemplo classe - Encapsulamento

```
public class Usuario{  
  
    private long idUsuario;  
    private String nome;  
    private String senha;  
  
    public void setNome(String nome){  
        this.nome = nome;  
    }  
    public String getNome(){  
        return this.nome;  
    }  
}
```



Uso do Lombok

@Data @Get @Set

```
public class Usuario{
```

```
    private long idUsuario;
```

```
    private String nome;
```

```
    private String senha;
```

```
public void setNome(String nome){
```

```
    this.nome = nome;
```

```
}
```

```
public String getNome(){
```

```
    return this.nome;
```

```
}
```



Exemplo classe - @Entity

A anotação **@Entity** pertence ao **JPA (Jakarta Persistence API)** e é usada no **Spring** para indicar que uma **classe Java representa uma tabela no banco de dados**.

@Data

@Entity

```
public class Usuario{  
  
    private long idUsuario;  
    private String nome;  
    private String senha;  
}
```

Exemplo classe - @Id

A anotação **@Id** no **JPA (Java Persistence API)** indica qual atributo da classe **é a chave primária da tabela no banco de dados**.

@Data

@Entity

```
public class Usuario{
```

@Id

```
private long idUsuario;
```

```
private String nome;
```

```
private String senha;
```

```
}
```



Exemplo classe - @GeneratedValue

Para **PostgreSQL**, a anotação @GeneratedValue normalmente usa a estratégia *SEQUENCE*, pois o PostgreSQL trabalha naturalmente com **seqüências** para gerar IDs.

@Data

@Entity

public class Usuario{

@Id

@GeneratedValue(strategy = GenerationType.SEQUENCE)

private long idUsuario;

private String nome;

private String senha;

}

Exemplo classe - @SequenceGenerator

A anotação `@SequenceGenerator` no **JPA** é usada para **definir uma sequência de geração de IDs**, normalmente utilizada com bancos como **PostgreSQL** e **Oracle**.

`@Data`

`@Entity`

`public class Usuario{`

`@Id`

`@GeneratedValue(strategy = GenerationType.SEQUENCE ,`

`generator = “usuario_seq”)`

`@SequenceGenerator(name = “usuario_seq”, sequenceName
= “usuario_sequence”, allocationSize = 1)`



Exemplo classe - @SequenceGenerator

```
private long idUsuario;  
private String nome;  
private String senha;  
}
```



Exemplo classe – Interface Repository

No **Spring Data JPA**, uma **interface** é usada para acessar os dados da **entidade** (`@Entity`) no banco de dados sem precisar escrever SQL manualmente. Isso é feito através do **JpaRepository**.

```
import  
org.springframework.data.jpa.repository.JpaRepository;  
  
public interface UsuarioRepository extends  
JpaRepository<Usuario, Long> {  
  
}
```



Exemplo classe – Interface Repository

No **Spring Data JPA**, uma **interface** é usada para acessar os dados da **entidade** (`@Entity`) no banco de dados sem precisar escrever SQL manualmente. Isso é feito através do **JpaRepository**.

```
import
org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends
JpaRepository<Usuario, Long> {

}
```

Exemplo classe – Interface Repository

No **Spring Data JPA**, uma **interface** é usada para acessar os dados da **entidade** (`@Entity`) no banco de dados sem precisar escrever SQL manualmente. Isso é feito através do **JpaRepository**.

```
import
org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends
JpaRepository<Usuario, Long> {

}
```

Application properties

Banco de Dados PostgreSQL

spring.datasource.driver-class-name=org.postgresql.Driver

spring.datasource.url=jdbc:postgresql://localhost:5432/ifms

spring.datasource.username=postgres

spring.datasource.password=ifms

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

spring.jpa.database=POSTGRESQL

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=update

#Formata a saída de LOG

spring.jpa.properties.hibernate.format_sql=true



UsuarioController- ENDPoint

```
@RestController
public class UsuarioController {

    @Autowired
    UsuarioRepository repository;

    @PostMapping("save")
    public Usuario save(@RequestBody Usuario usuario) {

        return repository.save(usuario);
    }
}
```



Pom.xml

```
<dependency>  
<groupId>org.postgresql</groupId>  
<artifactId>postgresql</artifactId>  
<scope>runtime</scope>  
</dependency>
```



Teste com Postmann

chrome web store

Discover Extensões Temas

🔍 Pesquisar extensões e temas

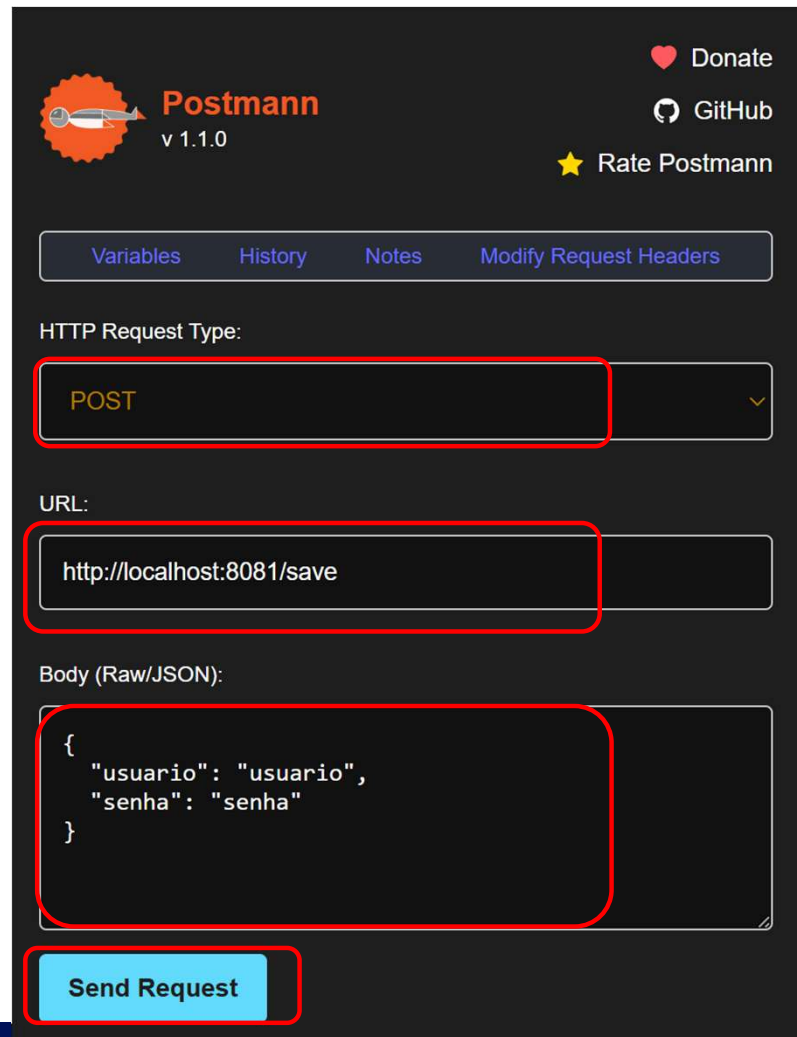
 **Postmann**

5,0 ★ (2 notas) ⓘ [Compartilhar](#)

Extensão Ferramentas para desenvolvedores 5.000 usuários

Remover do Chrome

Teste com Postmann



The screenshot shows the Postman v1.1.0 interface. At the top, there is a header with the Postman logo, version number, and links to 'Donate', 'GitHub', and 'Rate Postman'. Below the header, there are tabs for 'Variables', 'History', 'Notes', and 'Modify Request Headers'. The main section is titled 'HTTP Request Type:' and shows a dropdown menu with 'POST' selected. Below this, the 'URL:' field contains 'http://localhost:8081/save'. The 'Body (Raw/JSON):' section shows a JSON object:

```
{  "usuario": "usuario",  "senha": "senha"}
```

. At the bottom, there is a blue 'Send Request' button. Red rectangular boxes highlight the 'POST' dropdown, the URL field, the JSON body, and the 'Send Request' button.

Postmann v 1.1.0

Donate GitHub Rate Postmann

Variables History Notes Modify Request Headers

HTTP Request Type:

POST

URL:

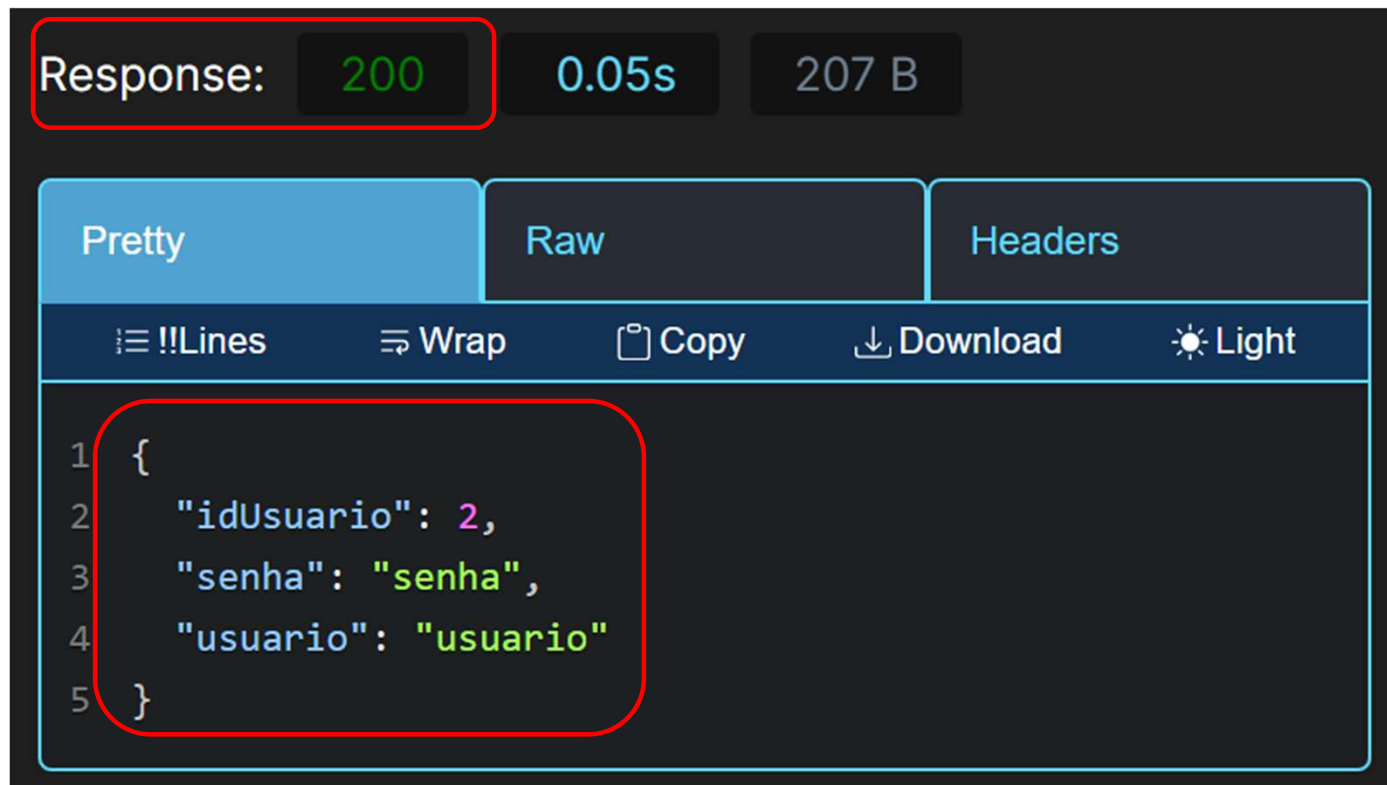
http://localhost:8081/save

Body (Raw/JSON):

```
{  "usuario": "usuario",  "senha": "senha"}
```

Send Request

Teste com Postmann



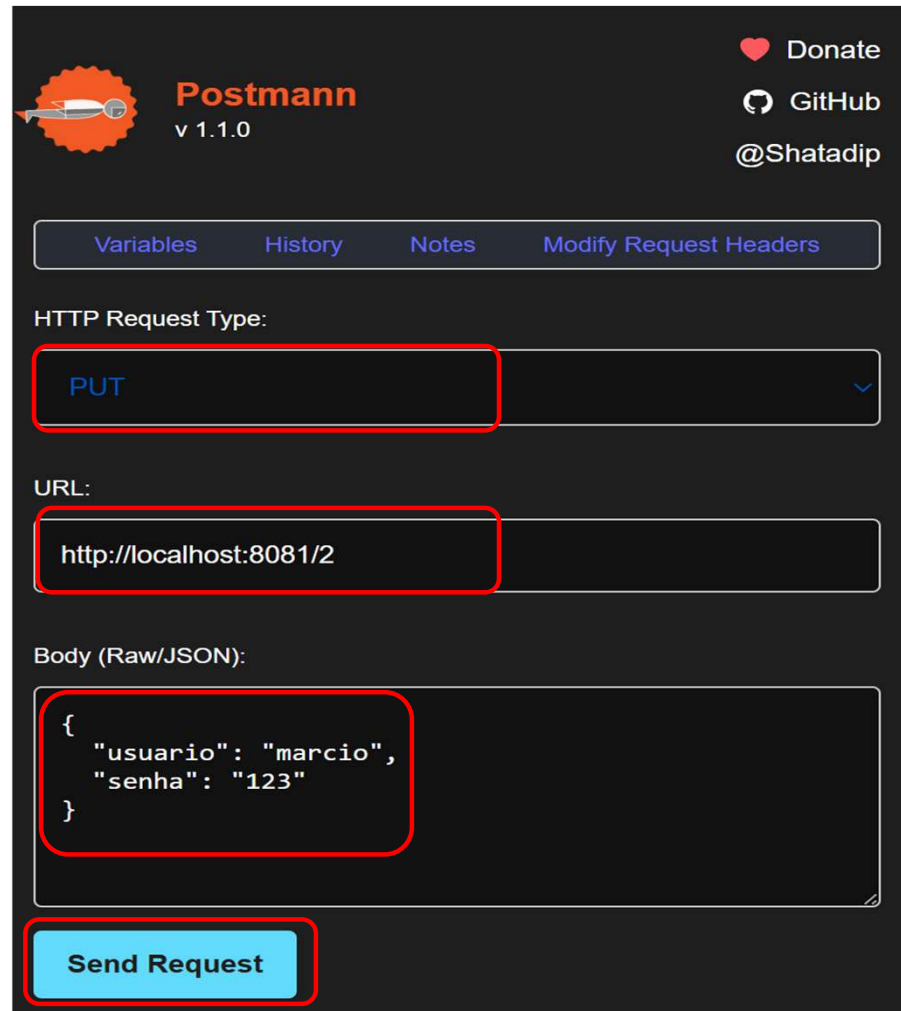
Update - PUT

Para elaborar o mecanismo de atualização de registros, basta construirmos:

```
@PutMapping("/{id}")  
public Usuario atualizar(@PathVariable long id, @RequestBody Usuario usuario) {  
  
    usuario.setIdUsuario(id);  
    repository.save(usuario);  
  
    return usuario;  
}
```



Put



The image shows the Postman v 1.1.0 interface. At the top, there is a header with the Postman logo, version number, and links to 'Donate', 'GitHub', and '@Shatadip'. Below the header, there are tabs for 'Variables', 'History', 'Notes', and 'Modify Request Headers'. The main section is titled 'HTTP Request Type:' and has a dropdown menu set to 'PUT'. Below this, the 'URL:' field contains 'http://localhost:8081/2'. The 'Body (Raw/JSON):' section contains a JSON object:

```
{  "usuario": "marcio",  "senha": "123"}
```

. At the bottom, there is a 'Send Request' button.

Postmann
v 1.1.0

Donate
GitHub
@Shatadip

Variables History Notes Modify Request Headers

HTTP Request Type:

PUT

URL:

http://localhost:8081/2

Body (Raw/JSON):

```
{  "usuario": "marcio",  "senha": "123"}
```

Send Request

Put

Response: 200 0.03s 196 B

Pretty Raw Headers

!!!Lines Wrap Copy Download Light

```
1 {
2   "idUsuario": 2,
3   "senha": "123",
4   "usuario": "marcio"
5 }
```

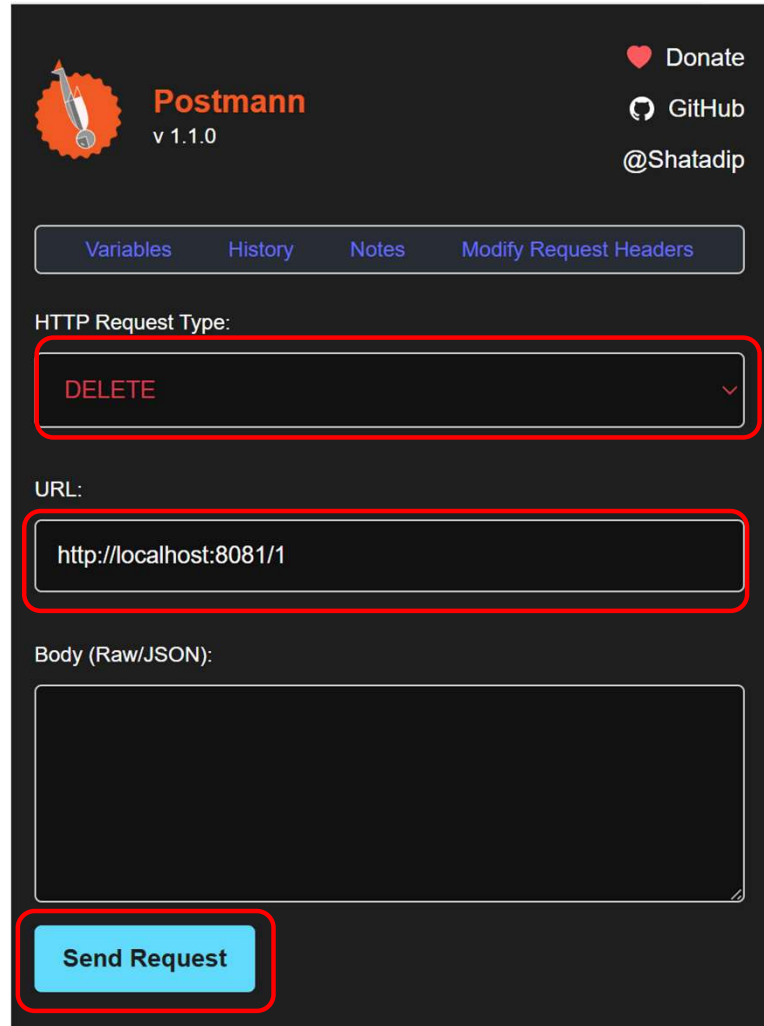


Delete

Para elaborar o mecanismo de remoção de registros, basta construirmos:

```
@DeleteMapping("/{id}")  
public String deletar(@PathVariable long id) {  
  
    repository.deleteById(id);  
  
    return "Usuário removido com sucesso";  
}
```

Delete



The image shows the Postman v 1.1.0 interface. At the top left is the Postman logo and version. At the top right are links to 'Donate', 'GitHub', and '@Shatadip'. Below these are tabs for 'Variables', 'History', 'Notes', and 'Modify Request Headers'. The 'HTTP Request Type' dropdown is set to 'DELETE'. The 'URL' field contains 'http://localhost:8081/1'. The 'Body (Raw/JSON)' field is empty. At the bottom is a 'Send Request' button. Red boxes highlight the 'DELETE' dropdown, the 'URL' field, and the 'Send Request' button.

Postmann
v 1.1.0

Donate
GitHub
@Shatadip

Variables History Notes Modify Request Headers

HTTP Request Type:

DELETE

URL:

http://localhost:8081/1

Body (Raw/JSON):

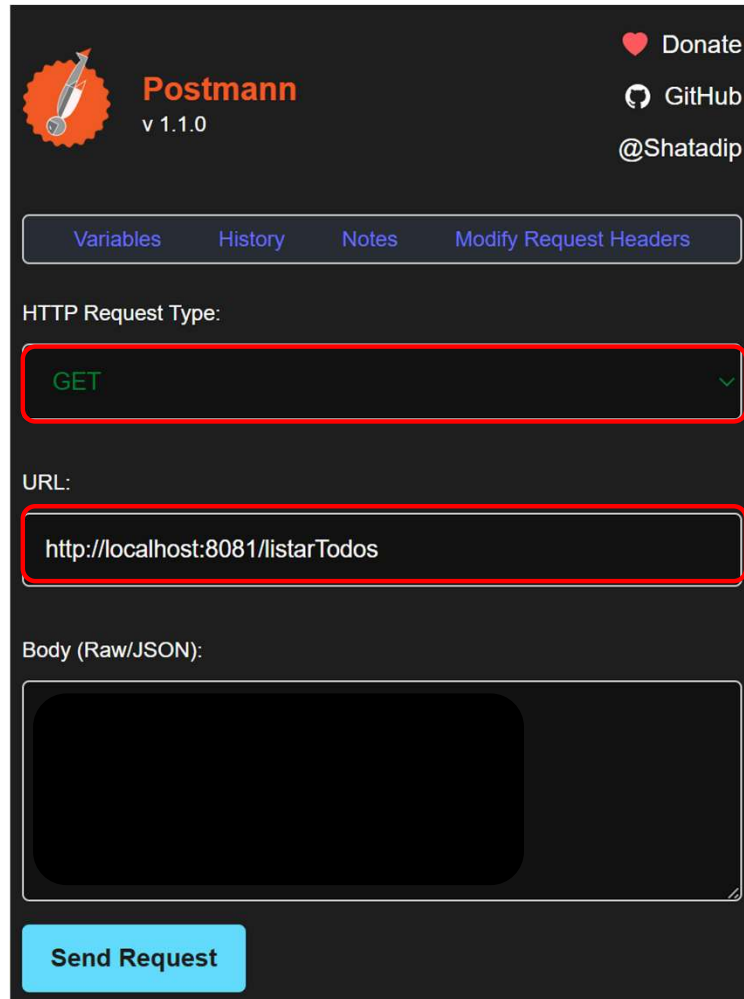
Send Request

GET

Para elaborar o mecanismo de listar todos os registros, basta construirmos:

```
@GetMapping("/listarTodos")  
public List<Usuario> listarTodos() {  
    return repository.findAll();  
}
```

GET



The image shows the Postman application interface for configuring an HTTP GET request. The top bar includes the Postman logo, version 1.1.0, and links to donate, GitHub, and the user @Shatadip. Below the top bar are tabs for Variables, History, Notes, and Modify Request Headers. The main configuration area is divided into three sections: HTTP Request Type, URL, and Body (Raw/JSON). The HTTP Request Type is set to GET. The URL is http://localhost:8081/listarTodos. The Body section is empty. A Send Request button is at the bottom.

Postmann
v 1.1.0

Donate
GitHub
@Shatadip

Variables History Notes Modify Request Headers

HTTP Request Type:

GET

URL:

http://localhost:8081/listarTodos

Body (Raw/JSON):

Send Request

GET

Response: 200 0.02s 305 B

Pretty Raw Headers

!!!Lines Wrap Copy Download Light

```
1 [
2   {
3     "idUsuario": 2,
4     "senha": "123",
5     "usuario": "marcio"
6   },
7   {
8     "idUsuario": 12,
9     "senha": "senha",
10    "usuario": "usuario"
11  },
12  {
13    "idUsuario": 13,
14    "senha": "senha",
15    "usuario": "usuario"
16  }
17 ]
```

GET – Buscar por nome de usuário

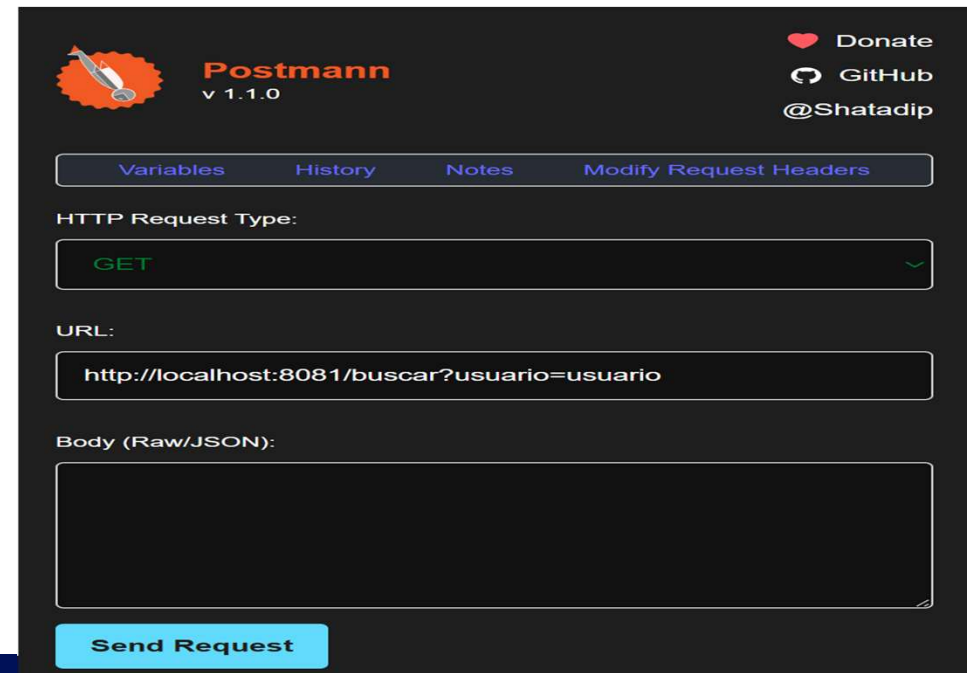
Para elaborar o mecanismo de busca por nome, basta construirmos:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  
  
    @Query("SELECT u FROM Usuario u WHERE u.usuario =:usuario")  
    Usuario buscarPorUsuario(@Param("usuario") String usuario);  
  
}
```

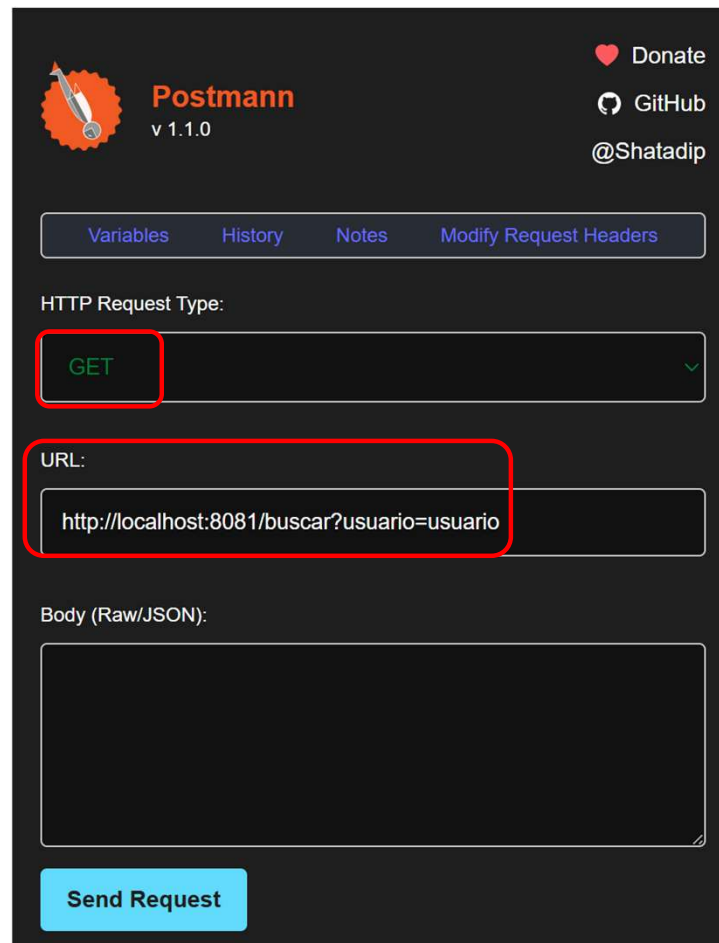
GET – Buscar por nome de usuário

Para elaborar o mecanismo de busca por nome, basta construirmos:

```
@GetMapping("/buscar")  
public List<Usuario> buscar(@RequestParam String usuario) {  
  
return repository.buscarPorUsuario(usuario);  
  
}
```



GET – Buscar por nome de usuário



The image shows the Postman application interface for configuring an HTTP request. At the top, the Postman logo and version (v 1.1.0) are visible on the left, and links to 'Donate', 'GitHub', and '@Shatadip' are on the right. Below the header, there are tabs for 'Variables', 'History', 'Notes', and 'Modify Request Headers'. The 'HTTP Request Type:' section shows 'GET' selected in a dropdown menu, which is highlighted with a red box. Below this, the 'URL:' section contains the text 'http://localhost:8081/buscar?usuario=usuario', also highlighted with a red box. The 'Body (Raw/JSON):' section is empty. At the bottom, there is a blue 'Send Request' button.

Postmann
v 1.1.0

Donate
GitHub
@Shatadip

Variables History Notes Modify Request Headers

HTTP Request Type:

GET

URL:

http://localhost:8081/buscar?usuario=usuario

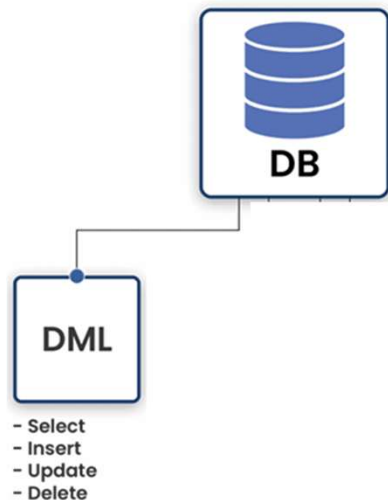
Body (Raw/JSON):

Send Request

Select

O comando `select` é utilizado para obter informação (individualmente ou múltiplas) de registros em tabelas previamente criadas no banco de dados, possibilitando obter as informações de forma estruturada e/ou personalizada:

Select *coluna* from *tabela* where *condição*



```
@Query("SELECT u FROM Usuario u WHERE u.idUsuario = :id")  
Usuario buscarPorId(@Param("id") Long id);
```

```
@Query("SELECT u.nome FROM Usuario u")  
List<Usuario> listarUsuario();
```

```
@Query("SELECT u.usuario, u.senha FROM Usuario u")  
List<Usuario> selecionaNomeSenha();
```

UsuarioController.java

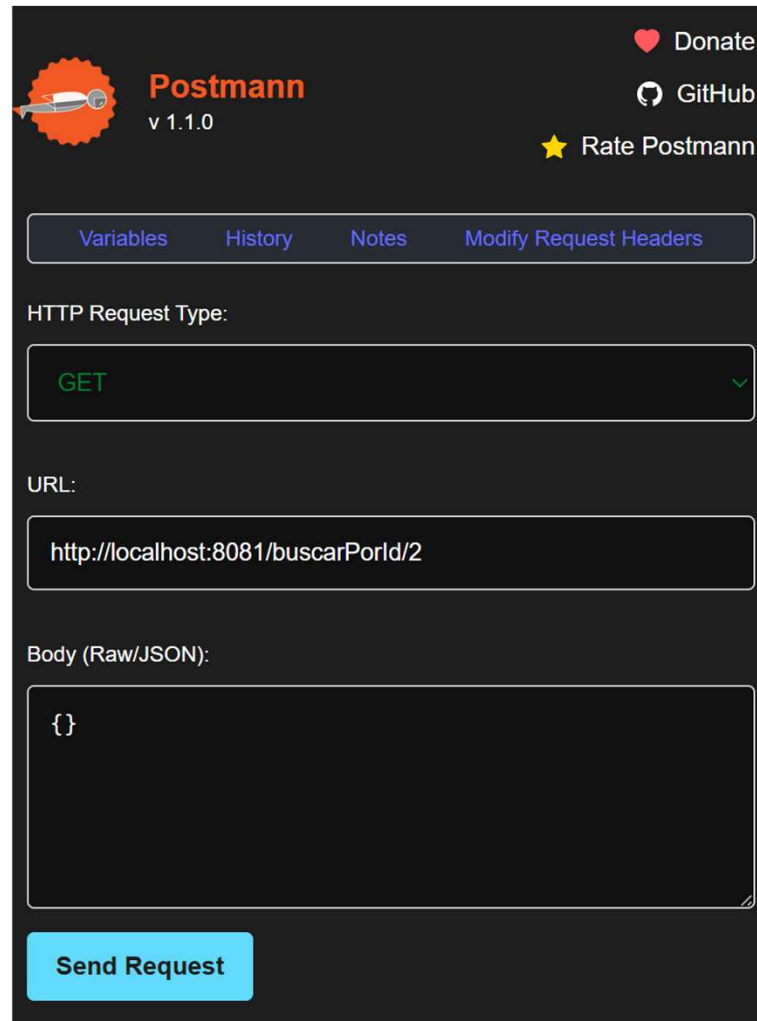
```
@GetMapping("/buscarPorId/{id}")  
public Usuario  
buscarPorId(@PathVariable Long id) {  
    return repository.buscarPorId(id);  
}
```

```
@GetMapping("/listarNomes")  
public List<Usuario>  
listarNomes() {  
    return repository.listarUsuario();  
}
```

```
@GetMapping("/listarNomeSenha")  
public List<Usuario> listarNomeSenha() {  
    return repository.selecionaNomeSenha();  
}
```



Usuario buscarPorId(@Param("id") Long id);



Postman v 1.1.0

Donate GitHub Rate Postmann

Variables History Notes Modify Request Headers

HTTP Request Type:

GET

URL:

http://localhost:8081/buscarPorId/2

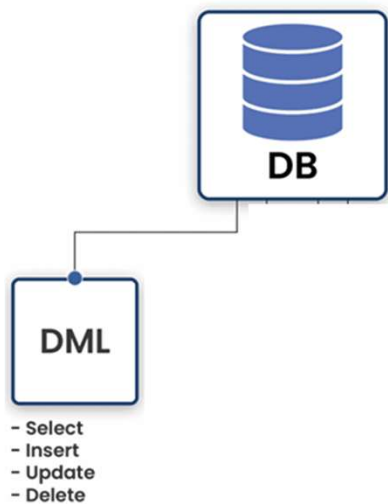
Body (Raw/JSON):

{}

Send Request

Banco de dados – Select personalizado

Uma das principais funções do `SELECT` é a possibilidade de se obter um conjunto de dados (extraídos do banco de dados) de maneira personalizada. Constantemente utilizamos operadores relacionais em nossas pesquisas.

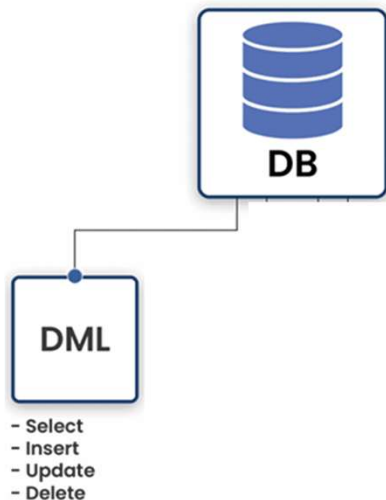


```
@Query("SELECT u FROM Usuario u WHERE u.idUsuario > 0")  
List<Usuario> usuariosExistentes();
```

=	igual a
>	maior do que
>=	maior ou igual a
<	menor do que
<=	menor ou igual a
<>	diferente de

Banco de dados – Select personalizado

Os **operadores lógicos** são usados em consultas SQL para **combinar condições** em cláusulas como `WHERE`, retornando valores **verdadeiro (TRUE)** ou **falso (FALSE)**. Eles permitem refinar e controlar a seleção de registros. Podemos utilizar **AND**, **OR** e **NOT**.



```
@Query("SELECT u.usuario, u.senha FROM Usuario u
WHERE u.usuario IS NOT NULL
AND u.usuario <> ''
AND u.senha IS NOT NULL
AND u.senha <> ''")
List<Usuario> usuarioSenhaNaoNuloNaoBranco();
```

Negação

P	$\sim P$
V	F
F	V

Conjunção

P	Q	$P \wedge Q$
V	V	V
V	F	F
F	V	F
F	F	F

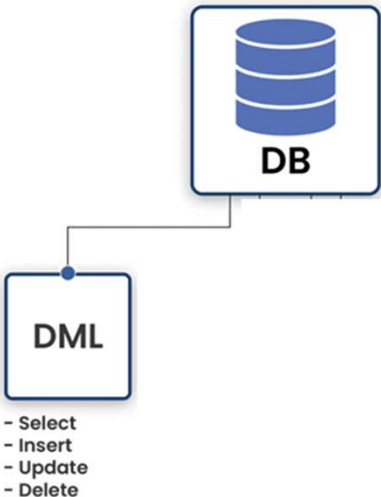
Disjunção

P	Q	$P \vee Q$
V	V	V
V	F	V
F	V	V
F	F	F



Banco de dados – Select personalizado Like

LIKE é um operador usado na cláusula WHERE para pesquisar padrões em colunas de texto:

	Expressão JPQL	Dado	Resultado
	u.nome LIKE 'Marcio%'	Marcio Silva	Qualquer string que inicie com "Marcio"
	u.nome LIKE '%Oliveira'	Marcio Silva Oliveira	Qualquer string que termine com "Oliveira"
	u.nome LIKE '%Silva%'	Marcio Silva Oliveira	Qualquer string que contenha "Silva"
	u.nome LIKE '%A_'	AO	Letra A na penúltima posição e mais 1 caractere no final
	u.nome LIKE '_A%'	MA	Letra A na segunda posição
	u.nome LIKE '_____%'	MARCIO	Qualquer string com pelo menos 3 caracteres

Banco de dados – Select personalizado Like

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE 'Marcio%")  
List<Usuario> buscarIniciaComMarcio();
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE '%Oliveira")  
List<Usuario> buscarTerminaComOliveira();
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE '%Silva%")  
List<Usuario> buscarContemSilva();
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE '%A_")  
List<Usuario> buscarPenultimaA();
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE '%_A")  
List<Usuario> buscarPenultimaA();
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE '____%")  
List<Usuario> buscarMinimoTresCaracteres();
```

Marcio Silva

Marcio Silva **Oliveira**

Marcio **Silva** Oliveira

AO

MA

MARCIO



Banco de dados – Select personalizado Like

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE CONCAT(:nome, '%')")  
List<Usuario> buscarIniciaCom(@Param("nome") String nome);
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE CONCAT('%',:nome)")  
List<Usuario> buscarTerminaComOliveira(@Param("nome") String nome);
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE CONCAT('%', :nome, '%')")  
List<Usuario> buscarContem(@Param("nome") String nome);
```

```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE CONCAT('%', :letra, '_')")  
List<Usuario> buscarPenultima(@Param("letra") String letra);
```

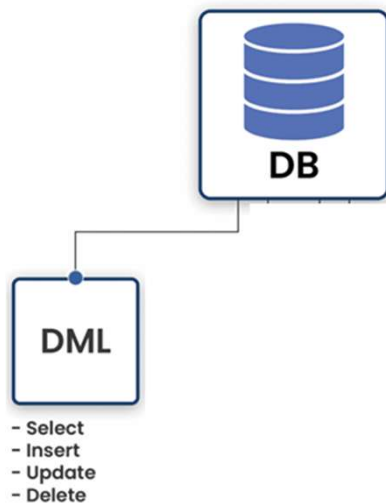
```
@Query("SELECT u FROM Usuario u WHERE u.nome LIKE CONCAT('%', :letra, '_')")  
List<Usuario> buscarPenultima(@Param("letra") String letra);
```



Banco de dados – Select/Funções agregadas

As funções de agregação SQL realizam cálculos em um conjunto de valores de uma coluna de uma tabela e retornam um único valor resumido. As funções mais comuns incluem COUNT (contagem de linhas), SUM (soma dos valores), AVG (média dos valores), MAX (maior valor) e MIN (menor valor).

Select FUNÇÃO(COLUNA) from TABELA

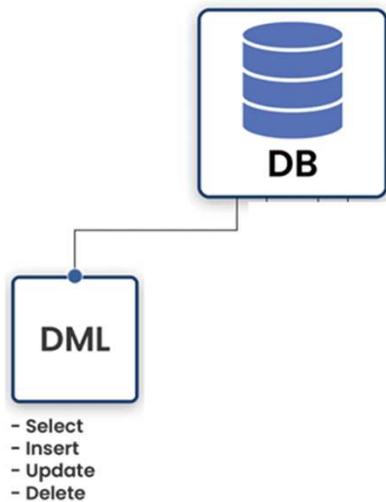


Banco de dados – Select/SUM

A função SQL SUM() é usada para calcular a soma total de valores numéricos numa coluna específica de uma tabela, retornando um único valor que representa a soma de todas as linhas que satisfazem os critérios da consulta

```
Select SUM(u.idusuario) from usuario u;
```

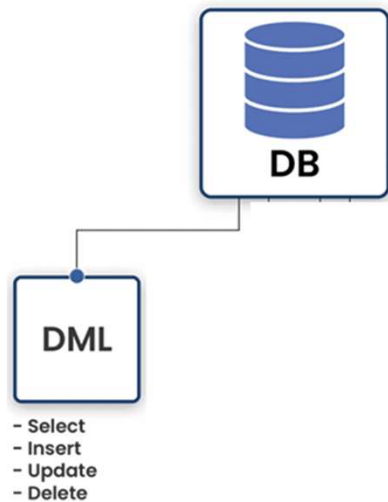
```
@Query("SELECT SUM(u.idUsuario) FROM Usuario u")  
Long somarIds();
```



Banco de dados – Select/COUNT

A função de agregação COUNT em SQL é usada para **contar o número de linhas** num conjunto de resultados ou que atendam a uma condição específica.

```
Select COUNT(u.nome) from usuário u;
```

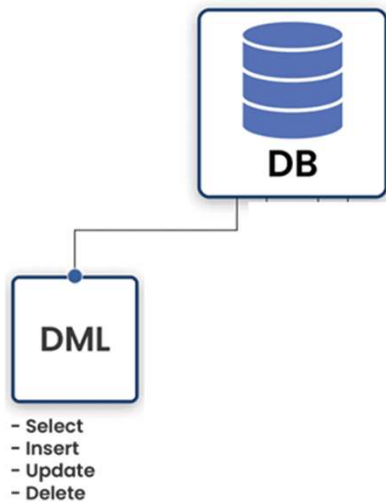


```
@Query("SELECT COUNT(u.nome) FROM Usuario u")  
Long contarNomes();
```


Banco de dados – Select/MIN

A função de agregação MIN() em SQL é usada para encontrar o **menor valor** dentro de uma coluna específica em uma tabela de banco de dados.

```
Select MIN(u.idusuario) from usuario u;
```

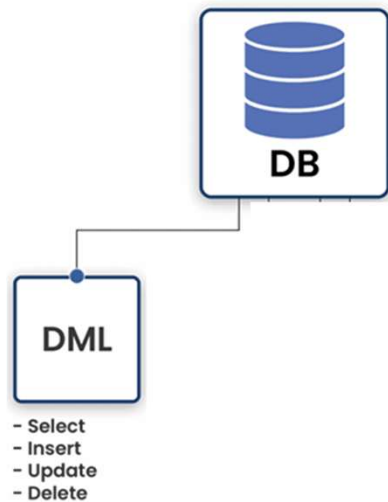


```
@Query("SELECT MIN(u.idusuario) FROM Usuario u")  
Long menorIdCadastrado();
```

Banco de dados – Select/MAX

A função de agregação MAX() no SQL serve para encontrar o **maior valor** dentro de um conjunto de dados em uma coluna específica.

```
Select MAX(u.idusuario) from usuario u;
```

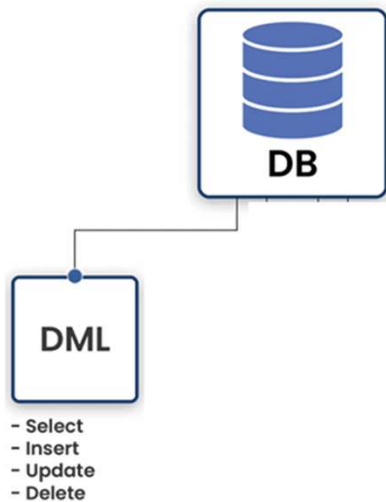


```
@Query("SELECT MAX(u.idUsuario) FROM Usuario u")  
Long maiorId();
```

Banco de dados – Select/AVG

A função de agregação AVG() no SQL calcula o **valor médio** de uma coluna numérica específica dentro de uma tabela

```
Select AVG(u.idusuario) from usuario u;
```



```
@Query("SELECT AVG(u.idusuario) FROM Usuario u")  
double mediaIds();
```